МЕТОДИЧЕСКИЕ УКАЗАНИЯ К ПРАКТИЧЕСКОМУ ЗАНЯТИЮ № 2

Тема: «Проектирование структур данных предметной области: схема сущностей, полей и формат обмена данными (JSON и JSON-Schema)»

Вариант А — базовая траектория · Вариант Б — продвинутая траектория

Параметр	Значение
Дисциплина	Программирование и прототипирование ИТ-решений
Направление подготовки	09.03.03 Прикладная информатика (профиль «Бизнес и системная аналитика»)
Общая трудоёмкость	2 пары (4 академических часа / 180 минут), разделённые на Этап 1 и Этап 2
Этап 1 (Пара 1 — 90 мин)	Проектирование модели сущностей (Data Dictionary, ER-диаграмма) и файловый обмен JSON (json.dump / json.load)
Этап 2 (Пара 2 — 90 мин)	Разработка контракта данных JSON-Schema, программирование валидатора и стресс-тесты на «грязных» данных
Формат работы	В парах с обязательной сменой ролей между Этапом 1 и Этапом 2 (Системный аналитик ↔ Разработчик)
Инструменты	VS Code, Python 3.11+ (модуль json), Markdown / Mermaid, GitHub (через веб-браузер)
Форма контроля	Единый отчёт пары по итогам 2 этапов + репозиторий GitHub с 4 файлами + устная защита

1. Цель работы

В практической работе № 1 вы научились делить логику программы на отдельные функции. Однако в реальных ИТ-системах сервисы не передают друг другу десятки разрозненных переменных (q_liq, w_pct, depth, status...). Вместо этого данные об объекте упаковывают в единую карточку объекта (в Python это вложенный словарь dict), а при сохранении на диск или передаче по сети записывают в универсальном текстовом формате JSON.

Работа разделена на два последовательных этапа по 90 минут (2 пары):

- **На Этапе 1 (Пара 1 — 90 минут)** вы переводите текст заказчика в таблицу атрибутов (Data Dictionary), строите ER-диаграмму связей сущностей в Mermaid, создаёте вложенную структуру словаря в Python (entity_model.py), сохраняете её в файл data.json и проверяете обратное чтение с диска.
- **На Этапе 2 (Пара 2 — 90 минут)** вы меняетесь ролями в паре! Новый Аналитик формализует строгий контракт данных на языке JSON-Schema (файл schema.json), а новый Разработчик пишет модуль программной верификации (validator.py), который проверяет прочитанный JSON-файл и отсекает «сломанные» пакеты данных.

Наглядный пример: путь данных через Этап 1 и Этап 2

Сквозная архитектура практической работы № 2 (Этап 1 + Этап 2):

```
# ЭТАП 1 (Пара 1): Упаковываем разрозненные переменные в словарь Python и сохраняем в JSON-файл
well_passport: dict = {
    "well_id": "W-115",
    "depth_m": 2250,
    "q_liq": 97.5,
    "w_pct": 42.0,
    "is_active": True,
    "pump": {"model": "ЭЦН5-80", "power_kw": 115.0}
}
# Сохраняем на диск в файл data.json -> теперь данные можно передать в другой отдел или сервис!

# ЭТАП 2 (Пара 2): Защищаем систему с помощью JSON-Schema и программного валидатора
# Если кто-то пришлёт в data.json глубину -100 м или забудет поле "well_id",
# наш валидатор сверит данные со схемой schema.json и выдаст точный список нарушений контракта!
```

2. Краткая шпаргалка (Глоссарий на обе пары)

Перед началом работы ознакомьтесь с ключевыми терминами. Первая половина таблицы пригодится на Этапе 1, вторая — на Этапе 2.

Термин	Простыми словами и на каком этапе используется
Сущность (Entity)	[Этап 1] Объект предметной области, о котором система хранит данные («Скважина», «Насос», «Пакет телеметрии», «Датчик»).
Словарь данных (Data Dictionary)	[Этап 1] Таблица аналитика, где перечислены все поля сущности, их типы данных, единицы измерения и примеры значений.
ER-диаграмма (Entity-Relationship)	[Этап 1] Схема связей между сущностями (например, «Одна скважина содержит один насос» 1:1 или «Один пакет содержит много датчиков» 1:N).
Словарь Python (dict)	[Этап 1] Структура данных в фигурных скобках {"ключ": значение}, хранящая карточку объекта в оперативной памяти Python.
JSON (JavaScript Object Notation)	[Этап 1] Текстовый формат обмена данными. Ключи всегда в двойных кавычках "...", а вместо True/False/None пишутся true/false/null.
Сериализация (json.dump / dumps)	[Этап 1] Запись словаря Python в файл .json (dump) или превращение в JSON-строку (dumps). Параметр ensure_ascii=False сохраняет кириллицу.
Десериализация (json.load / loads)	[Этап 1] Чтение текста из файла .json обратно в словарь Python для расчётов и проверок.
Контракт данных (Data Contract)	[Этап 2] Официальная договорённость между аналитиком и разработчиком: какие поля обязательны, какого они типа и в каких границах лежат.
JSON-Schema (файл schema.json)	[Этап 2] Стандартизированный JSON-документ («паспорт правил»), содержащий блоки type, required, properties, minimum, maximum, enum.
Валидатор (validator.py)	[Этап 2] Программный модуль, который проверяет входной словарь или JSON-файл по правилам схемы и возвращает список ошибок.

3. План работы

Обратите внимание на главное правило: на Паре 1 Студент 1 работает Системным аналитиком (CA), а Студент 2 — Разработчиком (Dev). На Паре 2 вы ОБЯЗАТЕЛЬНО меняетесь ролями (Студент 2 становится CA, а Студент 1 — Dev)

Время (Пара 1)	Шаг Этапа 1	Что делает пара на Первой паре (90 минут)
00–15 мин	Старт и расчёт варианта	Сложите ваши номера по журналу N = N1 + N2, посчитайте индивидуальные параметры скважины. Создайте папку prac2.
15–45 мин	Параллельная работа (Этап 1)	Аналитик (Студент 1) создаёт README.md: заполняет Data Dictionary и рисует ER-диаграмму Mermaid. Разработчик (Студент 2) пишет скрипт entity_model.py.
45–70 мин	Генерация и чтение data.json	Запустите python entity_model.py: программа создаст файл data.json, прочитает его обратно через json.load() и посчитает производный показатель.
70–90 мин	Челлендж Этапа 1	Выполните Челлендж № 1 (расширение структуры новым полем по просьбе заказчика) и покажите промежуточный результат преподавателю.

Время (Пара 2)	Шаг Этапа 2	Что делает пара на Второй паре (90 минут)
00–10 мин	Смена ролей за ПК!	Поменяйтесь местами! Теперь Студент 2 — Системный аналитик, а Студент 1 — Разработчик. Откройте папку prac2 с файлами первой пары.
10–40 мин	Параллельная работа (Этап 2)	Новый Аналитик создаёт файл контракта schema.json и дописывает раздел 2 в README.md. Новый Разработчик пишет модуль проверки validator.py.
40–65 мин	Прогон тестов валидации	Запустите python validator.py: проверьте эталонный файл data.json и 3 провокационных («грязных») набора данных. Заполните таблицу сверки.
65–80 мин	Челлендж Этапа 2	Выполните Челлендж № 2 («Поймай чужую ошибку»: взаимная проверка сломанного JSON-файла) и проверьте все 4 файла проекта.
80–90 мин	Загрузка на GitHub и сдача	Загрузите README.md, entity_model.py, data.json, schema.json и validator.py на GitHub через браузер (Раздел 8) и защитите работу.

Итоговый состав папки prac2 к концу второй пары (Чек-лист из 5 файлов):

- **1. README.md —** документация проекта с таблицей полей, ER-диаграммой Mermaid и таблицей сверки тестов валидации.
- **2. entity_model.py —** скрипт Этапа 1, формирующий карточку объекта, сохраняющий её в JSON и читающий обратно.
- **3. data.json —** сгенерированный программой файл с вашими индивидуальными данными в формате JSON.

- **4. schema.json —** формальная схема контракта данных на стандарте JSON-Schema (Этап 2).
- **5. validator.py —** скрипт Этапа 2, проверяющий данные по схеме и отлавливающий ошибки типов, границ и пропущенных полей.

4. Выбор траектории (Вариант А или Вариант Б) и расчёт индивидуальных чисел

Выберите один из двух вариантов и рассчитайте свои индивидуальные параметры по сумме номеров студентов пары в журнале N = N1 + N2. Эти числа вы будете использовать и на Первой, и на Второй паре!

4.1. Вариант А (Базовый): «Цифровой паспорт нефтяной скважины и погружного насоса»

Легенда заказчика (Ведущий инженер цеха добычи): «Ребята, сейчас наши операторы присылают данные о скважинах кто во что горазд: один пишет 'Скв 101', другой забывает глубину, а третий вместо обводнённости присылает текст 'двадцать процентов' — и наша база падает! Нам нужен единый цифровой стандарт (JSON-паспорт) карточки скважины. В карточке должны быть строго зафиксированы: код скважины well_id (непустая строка), глубина depth_m (целое число от 500 до 4500 м), суточный дебит жидкости q_liq (число от 0.0 до 1000.0 м³/сут), обводнённость w_pct (число от 0.0 до 100.0 %), статус в работе is_active (логическое да/нет) и вложенный объект насоса pump, внутри которого указаны модель model (непустая строка) и мощность power_kw (число от 10.0 до 500.0 кВт).»

Формулы индивидуального расчёта для Варианта А:

```
N        = N1 + N2                  # сумма номеров студентов пары по журналу (например, 5 + 10 =
15)
well_id  = f"W-{100 + N}"           # код скважины (при N = 15 будет "W-115")
depth_m  = 1500 + 50 * (N % 40)     # глубина скважины, м (целое число int, при N = 15 -> 2250)
q_liq    = 45.0 + 3.5 * (N % 30)    # дебит жидкости, м3/сут (float, при N = 15 -> 97.5)
w_pct    = 12.0 + 2.0 * (N % 35)    # обводнённость, % (float, при N = 15 -> 42.0)
power_kw = 40.0 + 5.0 * (N % 20)    # мощность насоса ЭЦН, кВт (float, при N = 15 -> 115.0)
```

4.2. Вариант Б (Продвинутый): «Пакет телеметрии куста скважин с массивом датчиков»

Легенда заказчика (Архитектор АСУ ТП): «Коллеги, мы проектируем протокол обмена данными между контроллером кустовой площадки и сервером. Каждую минуту контроллер формирует JSON-пакет телеметрии. В корне пакета должны быть: номер куста pad_id (строка вида 'PAD-XX'), код скважины well_id (строка), пластовое давление p_res_mpa (число от 5.0 до 60.0 МПа), забойное давление p_bhp_mpa (число от 1.0 до 60.0 МПа), режим работы mode (строго одно из трёх слов: 'FLOWING', 'PUMPING' или 'SHUT_IN') и массив датчиков sensors (список минимум из 2 объектов). Каждый датчик в списке sensors содержит: код датчика code (строка), показание value (число) и единицу измерения unit (строго из списка: 'MPa', 'C', 'm3/d'). Важное физическое правило: при режиме 'PUMPING' забойное давление p_bhp_mpa не может превышать пластовое p_res_mpa!»

Формулы индивидуального расчёта для Варианта Б:

```
N         = N1 + N2                   # сумма номеров студентов пары по журналу
pad_id    = f"PAD-{10 + (N % 20)}"    # идентификатор куста скважин
well_id   = f"W-{200 + N}"            # код скважины
```

```
p_res_mpa = 22.0 + 0.5 * (N % 20)     # пластовое давление, МПа
p_bhp_mpa = 12.0 + 0.4 * (N % 20)     # забойное давление, МПа
temp_c    = 45.0 + 1.5 * (N % 15)     # показание датчика температуры T-01, градусы C
rate_m3d  = 60.0 + 4.0 * (N % 25)     # показание датчика расхода Q-01, м3/сут
```

ЭТАП 1 (ПАРА 1 — 90 МИНУТ): ПРОЕКТИРОВАНИЕ МОДЕЛИ СУЩНОСТЕЙ И ФАЙЛОВЫЙ ОБМЕН JSON

Роли на Паре 1: Студент 1 — Системный аналитик (CA) · Студент 2 — Разработчик (Dev)

5. Выполнение Этапа 1 (Первая пара)

Создайте на компьютере рабочую папку prac2 и откройте её в Visual Studio Code (File -> Open Folder -> prac2). На первой паре ваша цель — спроектировать структуру сущностей в README.md и научить Python сохранять и читать ваш объект через файл data.json.

Шаг 1.1 (Делает Системный аналитик — Студент 1): Таблица полей (Data Dictionary) и ER-диаграмма

Создайте в папке prac2 файл README.md. Запишите в него заголовок проекта, ваши ФИО, расчёт индивидуальных чисел и заполните Таблицу структуры данных (выберите таблицу своего варианта А или Б):

Поле (ключ)	Описание смысла поля	Тип Python	Тип JSON	Обяз.?	Ваше значение (Пример)
well_id	Идентификатор (код) скважины	str	string	Да	"W-115" (подставьте свой)
depth_m	Глубина скважины, м	int	integer	Да	2250 (подставьте свой)
q_liq	Дебит жидкости, м³/сут	float	number	Да	97.5 (подставьте свой)
w_pct	Обводнённость продукции, %	float	number	Да	42.0 (подставьте свой)
is_active	Признак работы скважины	bool	boolean	Да	True (в JSON: true)
pump	Вложенный объект: погружной насос	dict	object	Да	{...}
pump.model	Модель насоса ЭЦН	str	string	Да	"ЭЦН5-80-1800"
pump.power_kw	Мощность насоса, кВт	float	number	Да	115.0 (подставьте свой)

Поле (путь	Описание смысла	Тип	Тип JSON	Обяз.?	Ваше значение

JSON)	поля (Вариант Б)	Python			(Пример)
pad_id	Номер кустовой площадки	str	string	Да	"PAD-15" (ваш расчёт)
well_id	Код скважины	str	string	Да	"W-215" (ваш расчёт)
p_res_mpa	Пластовое давление, МПа	float	number	Да	24.5 (ваш расчёт)
p_bhp_mpa	Забойное давление, МПа	float	number	Да	14.0 (ваш расчёт)
mode	Режим эксплуатации скважины	str	string	Да	"PUMPING"
sensors	Список замеров с датчиков	list[dict]	array	Да	[{...}, {...}]
sensors[].code	Код датчика в массиве	str	string	Да	"T-01", "Q-01"
sensors[].value	Измеренное показание датчика	float	number	Да	52.5 и 80.0 (ваши числа)
sensors[].unit	Единица измерения датчика	str	string	Да	"C", "m3/d"

Сразу под таблицей вставьте в README.md код ER-диаграммы сущностей на языке Mermaid (для своего варианта А или Б):

Коды ER-диаграмм Mermaid для вставки в README.md (выберите свой вариант):

````
# --- ДЛЯ ВАРИАНТА А (связь 1 к 1: Скважина содержит один Насос) ---
```mermaid
erDiagram
    WELL_PASSPORT ||--|| PUMP_UNIT : "содержит (pump)"
    WELL_PASSPORT {
        string well_id "Код скважины"
        integer depth_m "Глубина, м"
        number q_liq "Дебит жидкости, м3/сут"
        number w_pct "Обводненность, %"
        boolean is_active "Статус активности"
    }
    PUMP_UNIT {
        string model "Модель насоса ЭЦН"
        number power_kw "Мощность, кВт"
    }
```

# --- ДЛЯ ВАРИАНТА Б (связь 1 ко многим: Пакет содержит список Датчиков) ---
```mermaid
erDiagram
    TELEMETRY_PACKET ||--|{ SENSOR_ITEM : "включает массив (sensors)"
    TELEMETRY_PACKET {
        string pad_id "Код куста"
        string well_id "Код скважины"
        number p_res_mpa "Пластовое давление, МПа"
````
````

```
        number p_bhp_mpa "Забойное давление, МПа"
        string mode "Режим работы"
    }
    SENSOR_ITEM {
        string code "Код датчика"
        number value "Показание"
        string unit "Единица измерения"
    }
```

Зачем это делаем: ER-диаграмма (Entity-Relationship) мгновенно показывает разработчику структуру вложенности: если между блоками стоит символ ||--||, то внутри словаря лежит одиночный вложенный словарь dict; если стоит «воронья лапка» ||--|{, то внутри лежит список словарей list[dict].

Шаг 1.2 (Делает Разработчик — Студент 2): Скрипт entity_model.py (сериализация и чтение JSON)

Пока Аналитик оформляет README.md, Разработчик создаёт в папке prac2 файл entity_model.py. Скопируйте код своего варианта (А или Б) и ОБЯЗАТЕЛЬНО подставьте ваши индивидуальные числа из Раздела 4:

Код файла entity_model.py для Варианта А (Этап 1):

```
"""Этап 1 (Вариант А): Моделирование карточки скважины, сохранение и чтение JSON."""

import json
from typing import Any


def build_well_passport() -> dict[str, Any]:
    """Создаёт словарь паспорта скважины с индивидуальными данными пары."""
    # ПОДСТАВЬТЕ СЮДА ВАШИ ИНДИВИДУАЛЬНЫЕ ЧИСЛА ИЗ ПУНКТА 4.1!
    return {
        "well_id": "W-115",
        "depth_m": 2250,
        "q_liq": 97.5,
        "w_pct": 42.0,
        "is_active": True,
        "pump": {
            "model": "ЭЦН5-80-1800",
            "power_kw": 115.0,
        },
    }


def save_to_json(data: dict[str, Any], filename: str = "data.json") -> None:
    """Сериализует словарь Python в текстовый файл формата JSON."""
    with open(filename, "w", encoding="utf-8") as file:
        json.dump(data, file, ensure_ascii=False, indent=4)


def load_from_json(filename: str = "data.json") -> dict[str, Any]:
    """Десериализует (читает) JSON-файл с диска обратно в словарь Python."""
    with open(filename, "r", encoding="utf-8") as file:
        return json.load(file)
```

```
def print_passport_summary(passport: dict[str, Any]) -> None:
    """Выводит красивую сводку по прочитанному из JSON паспорту и считает дебит чистой нефти."""
    q_oil = passport["q_liq"] * (1.0 - passport["w_pct"] / 100.0)
    pump_info = passport["pump"]
    print("=" * 56)
    print(f"ПАСПОРТ СКВАЖИНЫ ИЗ JSON: {passport['well_id']}")
    print("=" * 56)
    print(f"Глубина:           {passport['depth_m']} м")
    print(f"Дебит жидкости:    {passport['q_liq']:.2f} м3/сут (обводнённость
{passport['w_pct']:.1f}%)")
    print(f"Расчёт чистой нефти: {q_oil:.2f} м3/сут")
    print(f"Насосная установка: {pump_info['model']} (мощность {pump_info['power_kw']} кВт)")
    print("=" * 56)


if __name__ == "__main__":
    # 1. Формируем словарь в памяти Python
    my_passport = build_well_passport()

    # 2. Сохраняем его в файл data.json (сериализация)
    save_to_json(my_passport, "data.json")
    print("[OK] Файл 'data.json' успешно записан на диск!")

    # 3. Читаем файл обратно с диска (десериализация) и проверяем, что данные не потерялись
    loaded_passport = load_from_json("data.json")
    assert loaded_passport == my_passport, "Ошибка: прочитанный словарь не совпадает с исходным!"
    print("[OK] Обратное чтение через json.load() прошло без потерь!
")

    # 4. Печатаем инженерную сводку по прочитанным данным
    print_passport_summary(loaded_passport)
```

Код файла entity_model.py для Варианта Б (Этап 1):

```
"""Этап 1 (Вариант Б): Моделирование пакета телеметрии куста, сохранение и чтение JSON."""

import json
from typing import Any


def build_telemetry_packet() -> dict[str, Any]:
    """Формирует пакет телеметрии со списком датчиков по индивидуальным числам пары."""
    # ПОДСТАВЬТЕ СЮДА ВАШИ ИНДИВИДУАЛЬНЫЕ ЧИСЛА ИЗ ПУНКТА 4.2!
    return {
        "pad_id": "PAD-15",
        "well_id": "W-215",
        "p_res_mpa": 24.5,
        "p_bhp_mpa": 14.0,
        "mode": "PUMPING",
        "sensors": [
            {"code": "T-01", "value": 52.5, "unit": "C"},
            {"code": "Q-01", "value": 80.0, "unit": "m3/d"},
        ],
    }
```

```
def save_to_json(data: dict[str, Any], filename: str = "data.json") -> None:
    """Сериализует пакет телеметрии в JSON-файл."""
    with open(filename, "w", encoding="utf-8") as file:
        json.dump(data, file, ensure_ascii=False, indent=4)


def load_from_json(filename: str = "data.json") -> dict[str, Any]:
    """Читает JSON-файл с диска обратно в словарь Python."""
    with open(filename, "r", encoding="utf-8") as file:
        return json.load(file)


def print_telemetry_summary(packet: dict[str, Any]) -> None:
    """Выводит сводку по пакету телеметрии и считает депрессию на пласт."""
    drawdown = packet["p_res_mpa"] - packet["p_bhp_mpa"]
    print("=" * 58)
    print(f"ПАКЕТ ТЕЛЕМЕТРИИ: Куст {packet['pad_id']} | Скважина {packet['well_id']}")
    print("=" * 58)
    print(f"Режим работы:      {packet['mode']}")
    print(f"Давления:          Рпл = {packet['p_res_mpa']} МПа, Рзаб = {packet['p_bhp_mpa']} МПа
(dP = {drawdown:.2f} МПа)")
    print(f"Опрошено датчиков: {len(packet['sensors'])} шт.")
    for s in packet["sensors"]:
        print(f"  -> Датчик {s['code']}: {s['value']} {s['unit']}")
    print("=" * 58)


if __name__ == "__main__":
    pkt = build_telemetry_packet()
    save_to_json(pkt, "data.json")
    print("[OK] Файл 'data.json' успешно записан на диск!")

    loaded_pkt = load_from_json("data.json")
    assert loaded_pkt == pkt, "Ошибка: данные при чтении JSON исказились!"
    print("[OK] Обратное чтение через json.load() прошло успешно!
")

    print_telemetry_summary(loaded_pkt)
```

Шаг 1.3 (Вместе): Запуск entity_model.py и исследование файла data.json

Откройте терминал в VS Code (Ctrl + `) и запустите скрипт первой пары:

```
python entity_model.py
```

Откройте появившийся в папке файл data.json и сравните его с кодом на Python. Запишите наблюдения: в файле data.json все строковые ключи взяты в двойные кавычки, а булево значение True записано маленькими буквами как true.

Шаг 1.4. Челлендж № 1 (в конце Первой пары — 15 минут): «Новое требование к модели данных»

Прежде чем завершить Первую пару, выполните мини-задание от заказчика:

- **Для Варианта А:** добавьте в карточку скважины новое текстовое поле "field_name" (название месторождения, например "Приобское"). Внесите его: 1) в таблицу полей и ER-диаграмму в README.md; 2) в словарь функции build_well_passport(); 3) в вывод функции print_passport_summary().
- **Для Варианта Б:** добавьте в корень пакета телеметрии булево поле "is_online": True (признак связи со станцией) и добавьте 3-й датчик в список sensors (датчик устьевого давления {"code": "P-01", "value": 14.0, "unit": "MPa"}). Внесите изменения в README.md и в entity_model.py.

Перезапустите python entity_model.py, чтобы файл data.json на диске обновился! На этом Первая пара успешно завершена.

ЭТАП 2 (ПАРА 2 — 90 МИНУТ): КОНТРАКТ ДАННЫХ JSON-SCHEMA И ПРОГРАММНЫЙ ВАЛИДАТОР

СМЕНА РОЛЕЙ! Теперь Студент 2 — Системный аналитик (CA) · Студент 1 — Разработчик (Dev)

6. Выполнение Этапа 2 (Вторая пара — со сменой ролей!)

В начале второй пары поменяйтесь местами за компьютером! На Первой паре вы создали эталонный файл data.json. Но что произойдёт, если завтра внешний сервис или стажёр пришлёт в систему файл data.json, где глубина отрицательная (-100 м), обводнённость равна 150%, а поле well_id вообще удалено? Чтобы программа не сломалась, Новый Аналитик сейчас создаст строгий паспорт правил — файл schema.json, а Новый Разработчик напишет защитный фильтр — скрипт validator.py.

Шаг 2.1 (Делает Новый Аналитик — Студент 2): Создание файла контракта schema.json

Создайте в папке prac2 отдельный файл schema.json и скопируйте в него формальную схему JSON-Schema для вашего варианта (не забудьте, что в схеме уже учтено новое поле field_name / is_online из Челленджа № 1!):

Содержимое файла schema.json для Варианта А (Этап 2):

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "title": "WellPassportContract",
  "description": "Контракт данных цифрового паспорта скважины (Вариант А)",
  "type": "object",
  "required": ["well_id", "field_name", "depth_m", "q_liq", "w_pct", "is_active", "pump"],
  "properties": {
    "well_id": {
      "type": "string",
      "minLength": 1,
      "description": "Уникальный код скважины"
    },
    "field_name": {
      "type": "string",
      "minLength": 1,
      "description": "Название месторождения (добавлено в Челлендже 1)"
    },
    "depth_m": {
```

```
      "type": "integer",
      "minimum": 500,
      "maximum": 4500
    },
    "q_liq": {
      "type": "number",
      "minimum": 0.0,
      "maximum": 1000.0
    },
    "w_pct": {
      "type": "number",
      "minimum": 0.0,
      "maximum": 100.0
    },
    "is_active": {
      "type": "boolean"
    },
    "pump": {
      "type": "object",
      "required": ["model", "power_kw"],
      "properties": {
        "model": { "type": "string", "minLength": 1 },
        "power_kw": { "type": "number", "minimum": 10.0, "maximum": 500.0 }
      }
    }
  }
}
```

Содержимое файла schema.json для Варианта Б (Этап 2):

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "title": "TelemetryPacketContract",
  "description": "Контракт данных пакета телеметрии куста скважин (Вариант Б)",
  "type": "object",
  "required": ["pad_id", "well_id", "p_res_mpa", "p_bhp_mpa", "mode", "is_online", "sensors"],
  "properties": {
    "pad_id": { "type": "string", "minLength": 1 },
    "well_id": { "type": "string", "minLength": 1 },
    "p_res_mpa": { "type": "number", "minimum": 5.0, "maximum": 60.0 },
    "p_bhp_mpa": { "type": "number", "minimum": 1.0, "maximum": 60.0 },
    "mode": {
      "type": "string",
      "enum": ["FLOWING", "PUMPING", "SHUT_IN"]
    },
    "is_online": {
      "type": "boolean"
    },
    "sensors": {
      "type": "array",
      "minItems": 2,
      "items": {
        "type": "object",
        "required": ["code", "value", "unit"],
        "properties": {
          "code": { "type": "string", "minLength": 1 },
          "value": { "type": "number" },
          "unit": { "type": "string", "enum": ["MPa", "C", "m3/d"] }
```

```
        }
      }
    }
  }
}
```

Зачем это делаем: Обратите внимание: мы вынесли JSON-Schema в настоящий отдельный файл schema.json! Сейчас скрипт Разработчика прочитает правила прямо из этого файла schema.json и применит их к данным из data.json.

Шаг 2.2 (Делает Новый Разработчик — Студент 1): Скрипт валидации validator.py

Создайте в папке prac2 файл validator.py и скопируйте в него код своего варианта (А или Б). Этот скрипт загружает с диска оба файла — schema.json и data.json — и автоматически проверяет данные по правилам вашей схемы:

Код файла validator.py для Варианта А (Этап 2):

```
"""Этап 2 (Вариант А): Валидация данных data.json по правилам контракта schema.json."""

import copy
import json
from typing import Any


def load_json_file(filename: str) -> dict[str, Any]:
    """Загружает любой JSON-файл с диска в словарь Python."""
    with open(filename, "r", encoding="utf-8") as file:
        return json.load(file)


def validate_by_schema(data: dict[str, Any], schema: dict[str, Any]) -> tuple[bool, list[str]]:
    """Проверяет словарь data по правилам, прочитанным из файла schema.json.

    Args:
        data (dict[str, Any]): Проверяемая карточка скважины.
        schema (dict[str, Any]): Словарь схемы JSON-Schema.

    Returns:
        tuple[bool, list[str]]: (True/False, список найденных нарушений контракта).
    """
    errors: list[str] = []
    req_fields: list[str] = schema.get("required", [])
    props: dict[str, Any] = schema.get("properties", {})

    # 1. Проверка наличия всех обязательных ключей из списка required
    for field in req_fields:
        if field not in data:
            errors.append(f"Отсутствует обязательное поле '{field}'")
    if errors:
        return False, errors

    # 2. Проверка строковых полей (minLength)
    for str_field in ("well_id", "field_name"):
        if str_field in data:
```

```
            if not isinstance(data[str_field], str) or len(data[str_field].strip()) < 1:
                errors.append(f"Поле '{str_field}' должно быть непустой строкой")

    # 3. Проверка целочисленной глубины depth_m по границам из schema.json
    d_rules = props["depth_m"]
    depth = data["depth_m"]
    if not isinstance(depth, int) or isinstance(depth, bool):
        errors.append("Поле 'depth_m' должно быть целым числом (integer)")
    elif not (d_rules["minimum"] <= depth <= d_rules["maximum"]):
        errors.append(
            f"Глубина depth_m={depth} вне границ схемы [{d_rules['minimum']};
{d_rules['maximum']}]"
        )

    # 4. Проверка числовых полей q_liq и w_pct по границам из schema.json
    for num_field in ("q_liq", "w_pct"):
        rules = props[num_field]
        val = data[num_field]
        if not isinstance(val, (int, float)) or isinstance(val, bool):
            errors.append(f"Поле '{num_field}' должно быть числом (number)")
        elif not (rules["minimum"] <= float(val) <= rules["maximum"]):
            errors.append(
                f"Поле {num_field}={val} вне границ схемы [{rules['minimum']};
{rules['maximum']}]"
            )

    # 5. Проверка булева поля is_active
    if not isinstance(data["is_active"], bool):
        errors.append("Поле 'is_active' должно быть логическим типом (boolean)")

    # 6. Проверка вложенного объекта pump
    pump = data["pump"]
    pump_schema = props["pump"]
    if not isinstance(pump, dict):
        errors.append("Поле 'pump' должно быть вложенным словарём (object)")
    else:
        for p_req in pump_schema["required"]:
            if p_req not in pump:
                errors.append(f"Во вложенном блоке 'pump' отсутствует поле '{p_req}'")
        if "model" in pump and (not isinstance(pump["model"], str) or not pump["model"].strip()):
            errors.append("Поле 'pump.model' должно быть непустой строкой")
        if "power_kw" in pump:
            p_rules = pump_schema["properties"]["power_kw"]
            pw = pump["power_kw"]
            if not isinstance(pw, (int, float)) or isinstance(pw, bool):
                errors.append("Поле 'pump.power_kw' должно быть числом")
            elif not (p_rules["minimum"] <= float(pw) <= p_rules["maximum"]):
                errors.append(
                    f"Мощность pump.power_kw={pw} вне границ [{p_rules['minimum']};
{p_rules['maximum']}]"
                )

    return len(errors) == 0, errors


if __name__ == "__main__":
    schema_doc = load_json_file("schema.json")
    valid_data = load_json_file("data.json")

    print("=" * 64)
```

```
    print("ЗАПУСК ПРОВЕРКИ КОНТРАКТА ДАННЫХ ПО ФАЙЛУ schema.json (Вариант А)")
    print("=" * 64)

    # Сценарий 1: Проверка нашего эталонного файла data.json
    ok1, errs1 = validate_by_schema(valid_data, schema_doc)
    print(f"
[Сценарий 1] Эталонный файл data.json -> {'УСПЕШНО (VALID)' if ok1 else 'ОШИБКА'}")

    # Сценарий 2: Выход за физические границы (глубина -100 м и обводнённость 145%)
    bad_limits = copy.deepcopy(valid_data)
    bad_limits["depth_m"] = -100
    bad_limits["w_pct"] = 145.0
    ok2, errs2 = validate_by_schema(bad_limits, schema_doc)
    print(f"
[Сценарий 2] Нарушение границ (depth=-100, w=145) -> {'УСПЕШНО' if ok2 else 'ОТКЛОНЕНО'}")
    for e in errs2:
        print(f"  x {e}")

    # Сценарий 3: Потеря обязательного поля well_id
    missing_key = copy.deepcopy(valid_data)
    del missing_key["well_id"]
    ok3, errs3 = validate_by_schema(missing_key, schema_doc)
    print(f"
[Сценарий 3] Удалено обязательное поле 'well_id' -> {'УСПЕШНО' if ok3 else 'ОТКЛОНЕНО'}")
    for e in errs3:
        print(f"  x {e}")

    # Сценарий 4: Ошибка типа во вложенном объекте pump (текст вместо числа мощности)
    bad_type = copy.deepcopy(valid_data)
    bad_type["pump"]["power_kw"] = "сто пятнадцать"
    ok4, errs4 = validate_by_schema(bad_type, schema_doc)
    print(f"
[Сценарий 4] Текст вместо числа в pump.power_kw -> {'УСПЕШНО' if ok4 else 'ОТКЛОНЕНО'}")
    for e in errs4:
        print(f"  x {e}")
```

Код файла validator.py для Варианта Б (Этап 2):

```
"""Этап 2 (Вариант Б): Валидация пакета data.json по контракту schema.json."""

import copy
import json
from typing import Any


def load_json_file(filename: str) -> dict[str, Any]:
    """Загружает JSON-файл с диска в словарь Python."""
    with open(filename, "r", encoding="utf-8") as file:
        return json.load(file)


def validate_by_schema(packet: dict[str, Any], schema: dict[str, Any]) -> tuple[bool, list[str]]:
    """Проверяет пакет телеметрии по правилам из файла schema.json и бизнес-логике."""
    errors: list[str] = []
    req_fields: list[str] = schema.get("required", [])
    props: dict[str, Any] = schema.get("properties", {})
```

```
    # 1. Проверка обязательных полей верхнего уровня
    for field in req_fields:
        if field not in packet:
            errors.append(f"Отсутствует обязательное поле '{field}'")
    if errors:
        return False, errors

    # 2. Проверка давлений по границам из schema.json
    for p_field in ("p_res_mpa", "p_bhp_mpa"):
        rules = props[p_field]
        val = packet[p_field]
        if not isinstance(val, (int, float)) or isinstance(val, bool):
            errors.append(f"Поле '{p_field}' должно быть числом")
        elif not (rules["minimum"] <= float(val) <= rules["maximum"]):
            errors.append(
                f"Давление {p_field}={val} вне границ схемы [{rules['minimum']};
{rules['maximum']}]"
            )

    # 3. Проверка режима работы по списку enum из schema.json
    allowed_modes = props["mode"]["enum"]
    if packet["mode"] not in allowed_modes:
        errors.append(f"Режим mode='{packet['mode']}' не входит в enum {allowed_modes}")

    if not isinstance(packet["is_online"], bool):
        errors.append("Поле 'is_online' должно быть логическим типом (boolean)")

    # Перекрёстное физическое правило
    if isinstance(packet["p_res_mpa"], (int, float)) and isinstance(packet["p_bhp_mpa"], (int,
float)):
        if packet["mode"] == "PUMPING" and float(packet["p_bhp_mpa"]) >
float(packet["p_res_mpa"]):
            errors.append("Физическое противоречие: при режиме PUMPING Рзаб > Рпл!")

    # 4. Проверка массива датчиков sensors (minItems и enum единиц измерения)
    sensors_schema = props["sensors"]
    min_items = sensors_schema["minItems"]
    allowed_units = sensors_schema["items"]["properties"]["unit"]["enum"]
    sensors = packet["sensors"]

    if not isinstance(sensors, list) or len(sensors) < min_items:
        errors.append(f"Массив 'sensors' должен содержать минимум {min_items} датчика")
    else:
        for idx, item in enumerate(sensors):
            if not isinstance(item, dict):
                errors.append(f"Датчик #{idx} должен быть объектом (dict)")
                continue
            for s_req in sensors_schema["items"]["required"]:
                if s_req not in item:
                    errors.append(f"В датчике #{idx} отсутствует поле '{s_req}'")
            if "unit" in item and item["unit"] not in allowed_units:
                errors.append(f"Датчик #{idx}: единица '{item['unit']}' не входит в enum
{allowed_units}")
            if "value" in item and (not isinstance(item["value"], (int, float)) or
isinstance(item["value"], bool)):
                errors.append(f"Датчик #{idx}: поле 'value' должно быть числом")

    return len(errors) == 0, errors
```

```
if __name__ == "__main__":
    schema_doc = load_json_file("schema.json")
    valid_pkt = load_json_file("data.json")

    print("=" * 66)
    print("ЗАПУСК ПРОВЕРКИ КОНТРАКТА ДАННЫХ ПО ФАЙЛУ schema.json (Вариант Б)")
    print("=" * 66)

    # Сценарий 1: Эталонный пакет из data.json
    ok1, errs1 = validate_by_schema(valid_pkt, schema_doc)
    print(f"
[Сценарий 1] Эталонный файл data.json -> {'УСПЕШНО (VALID)' if ok1 else 'ОШИБКА'}")

    # Сценарий 2: Нарушение enum режима и единицы измерения датчика
    bad_enum = copy.deepcopy(valid_pkt)
    bad_enum["mode"] = "OVERDRIVE"
    bad_enum["sensors"][0]["unit"] = "psi"
    ok2, errs2 = validate_by_schema(bad_enum, schema_doc)
    print(f"
[Сценарий 2] Нарушение списков enum -> {'УСПЕШНО' if ok2 else 'ОТКЛОНЕНО'}")
    for e in errs2:
        print(f"  x {e}")

    # Сценарий 3: Слишком мало датчиков в массиве (меньше minItems: 2)
    too_few = copy.deepcopy(valid_pkt)
    too_few["sensors"] = [valid_pkt["sensors"][0]]
    ok3, errs3 = validate_by_schema(too_few, schema_doc)
    print(f"
[Сценарий 3] Только 1 датчик в массиве sensors -> {'УСПЕШНО' if ok3 else 'ОТКЛОНЕНО'}")
    for e in errs3:
        print(f"  x {e}")

    # Сценарий 4: Физическое противоречие давлений при PUMPING (Рзаб > Рпл)
    bad_phys = copy.deepcopy(valid_pkt)
    bad_phys["mode"] = "PUMPING"
    bad_phys["p_bhp_mpa"] = 50.0
    bad_phys["p_res_mpa"] = 20.0
    ok4, errs4 = validate_by_schema(bad_phys, schema_doc)
    print(f"
[Сценарий 4] Противоречие давлений (Рзаб > Рпл) -> {'УСПЕШНО' if ok4 else 'ОТКЛОНЕНО'}")
    for e in errs4:
        print(f"  x {e}")
```

Шаг 2.3 (Вместе): Запуск валидатора и заполнение Таблицы сверки в README.md

Запустите в терминале скрипт второй пары:

```
python validator.py
```

Убедитесь, что Сценарий 1 прошёл со статусом УСПЕШНО (VALID), а провокационные Сценарии 2, 3 и 4 были отклонены с точным описанием причины. Заполните в README.md итоговую таблицу сверки:

Тестовый сценарий (Этап	Ожидание по	Фактический ответ	Совпало?

2)	schema.json	validator.py	
Сценарий 1: Эталонный файл data.json	VALID (0 ошибок)	(запишите вывод программы)	Да / Нет
Сценарий 2: Выход за границы / неверный enum	ОТКЛОНЕНО (2 ошибки)	(скопируйте тексты ошибок)	Да / Нет
Сценарий 3: Потеря обязательного поля / minItems	ОТКЛОНЕНО (1 ошибка)	(скопируйте текст ошибки)	Да / Нет
Сценарий 4: Ошибка типа во вложенном блоке / физика	ОТКЛОНЕНО (1 ошибка)	(скопируйте текст ошибки)	Да / Нет
Сценарий 5 (из Челленджа № 2): «Диверсия напарника»	ОТКЛОНЕНО	(какую ошибку спрятал напарник)	Да / Нет

Шаг 2.4. Челлендж № 2 (в конце Второй пары — 15 минут): «Слепая проверка: найди диверсию в JSON»

Проведите интерактивную игру между Аналитиком и Разработчиком:

- **Ход Аналитика:** пока Разработчик отвернулся на 1 минуту, Аналитик создаёт копию файла data.json под именем broken_data.json и вносит туда ровно 2 скрытых нарушения контракта (например, меняет число на строку, ставит глубину 9000 м или удаляет поле внутри насоса/датчика).
- **Ход Разработчика:** не открывая файл broken_data.json глазами, Разработчик вызывает в конце validator.py проверку load_json_file("broken_data.json") и по выводу программы в терминале угадывает, какие именно 2 поля испортил Аналитик! Запишите результат в 5-ю строку таблицы сверки.

7. Как загрузить все файлы двух этапов на GitHub и сдать отчёт

В конце Второй пары загрузите все 5 файлов из папки prac2 (README.md, entity_model.py, data.json, schema.json, validator.py) в ваш репозиторий на GitHub через браузер:

- **Шаг 7.1. Создайте репозиторий:** Откройте github.com -> нажмите «+» -> New repository -> введите имя prac2-json-contract -> выберите Private (или Public по указанию преподавателя) -> нажмите Create repository.
- **Шаг 7.2. Загрузите 5 файлов:** Нажмите uploading an existing file и перетащите мышкой все 5 файлов из папки prac2 в окно браузера.
- **Шаг 7.3. Сделайте Commit:** В поле комментария введите: Выполнена ПР №2 (Этап 1 + Этап 2: JSON и JSON-Schema) и нажмите зелёную кнопку Commit changes.
- **Шаг 7.4. Скопируйте ссылку:** Проверьте, что на главной странице репозитория отобразились таблицы и ER-диаграмма Mermaid из README.md, и скопируйте ссылку в отчёт.

Структура единого отчёта пары (по итогам двух пар):

- Титульный блок: ФИО обоих студентов, указание ролей на Этапе 1 (Пара 1) и после смены ролей на Этапе 2 (Пара 2), расчёт N = N1 + N2 и ссылка на GitHub.

- Раздел Этапа 1: таблица полей (Data Dictionary), ER-диаграмма сущностей, код entity_model.py, содержимое файла data.json и скриншот запуска Первой пары.
- Раздел Этапа 2: полный текст схемы schema.json, код валидатора validator.py, таблица сверки из 5 сценариев и скриншот запуска Второй пары.
- Письменные ответы на контрольные вопросы (по 2 вопроса к каждому этапу).

8. Контрольные вопросы для защиты и частые ошибки

Вопросы по Этапу 1 (Пара 1 — Моделирование и JSON):

- **Вопрос 1.1:** Почему передавать карточку скважины в виде единого словаря dict удобнее и надёжнее, чем передавать в функцию 8 отдельных переменных через запятую?
- **Вопрос 1.2:** Назовите минимум 3 отличия синтаксиса файла data.json от словаря dict в Python. Зачем в функции json.dump() нужен параметр ensure_ascii=False?

Вопросы по Этапу 2 (Пара 2 — Контракт данных и JSON-Schema):

- **Вопрос 2.1:** Что такое контракт данных (JSON-Schema) и за что отвечают ключевые слова required, properties, minimum/maximum и enum в файле schema.json?
- **Вопрос 2.2:** Почему в функции валидации в Python мы пишем проверку not isinstance(depth, bool) после isinstance(depth, int)? (Подсказка: каким типом в Python является логическое значение True/False?).

Частые ошибки на Этапе 1 и Этапе 2 (Проверьте себя!)

Симптом / Ошибка	Причина и как быстро исправить
[Этап 1] В файле data.json вместо кириллицы записаны коды \u041f\u0440\u0438	Забыли передать параметр ensure_ascii=False в функцию json.dump(data, file, ensure_ascii=False, indent=4).
[Этап 1] После Челленджа № 1 в файле data.json нет нового поля field_name / is_online	Вы добавили поле в код entity_model.py, но забыли повторно запустить python entity_model.py в терминале для перезаписи файла.
[Этап 2] При запуске validator.py возникает ошибка json.decoder.JSONDecodeError в schema.json	Проверьте синтаксис файла schema.json: в JSON нельзя ставить запятую после последнего элемента перед закрывающей скобкой } или] и нельзя писать комментарии через # или //.
[Этап 2] При изменении тестового словаря bad_limits портится исходный словарь valid_data	Вложенные словари (pump или sensors) копируются по ссылке! Используйте глубокое копирование copy.deepcopy(valid_data), как показано в шаблоне validator.py.